



LISTA DE EXERCÍCIOS - SISTEMAS INTELIGENTES APLICADOS

Lista de Exercícios 10

K-Means

DOCENTE: Thiago França Naves

DATA: 30/09/2026

ALUNO: Mayara Rodrigues Pereira e Leonardo José Reis Pinto

A questão 1 foi feita no Weka 3.8.6 com o SimpleKMeans. A base Water Treatment Plant foi baixada da UCI e convertida para .arff pelo script `converter_water.py`. As questões 2 e 3 estão no script `solucao.py`, que implementa o K-Means e mostra os centroides em cada iteração. Os dois estão no Apêndice A.

Questão 1

No Weka na aba de Clusters selecione o algoritmo K-Means. Faça testes com o algoritmo com a base Iris, Diabetes e execute testes com o algoritmo e teste diferentes valores de K, por fim discorra sobre os resultados e quantidade de clusters agrupados. Por fim, faça um último teste com a base Water Treatment Plant (UCI). Por ser uma base não supervisionada quais análises você pode tirar dos resultados da aplicação do k-means sobre essa base? Qual o melhor valor de k você encontrou?

Resposta. O SimpleKMeans usa distância euclidiana com os atributos normalizados de 0 a 1 e sorteia os centroides iniciais (semente 10, o padrão). Na Iris e na Diabetes usamos o modo “Classes to clusters evaluation”: a classe não participa do agrupamento, mas no final o Weka associa cada grupo à classe mais comum nele e conta quantas instâncias ficaram no grupo errado. O SSE é a soma das distâncias ao quadrado de cada ponto até o seu centroide.

Tabela 1. SimpleKMeans com diferentes valores de K.

Iris (3 classes)			Diabetes (2 classes)		
K	SSE	Mal agrupadas	K	SSE	Mal agrupadas
2	12,14	50 (33,3%)	2	121,26	255 (33,2%)
3	7,00	17 (11,3%)	3	105,97	371 (48,3%)
4	5,53	44 (29,3%)	4	95,24	369 (48,0%)
5	5,13	48 (32,0%)	5	87,18	417 (54,3%)

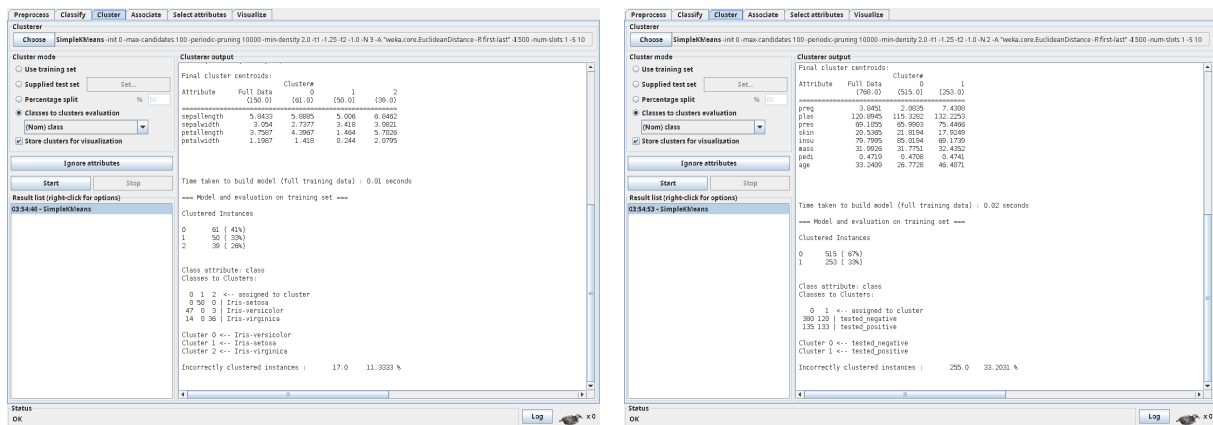


Figura 1. Iris com K = 3 (esquerda) e Diabetes com K = 2 (direita).

Na **Iris**, K = 3 é o que mais combina com as espécies. Com K = 3 os grupos ficaram com 61, 50 e 39 flores, e a matriz “classes to clusters” mostra que as 50 setosas caíram todas no mesmo grupo. Os 17 erros vêm de 14 virginicas que foram parar no grupo da versicolor e 3 versicolores no da virginica. Com K = 2 a setosa fica num grupo e as outras duas espécies juntas no outro, o que faz sentido geometricamente, já que versicolor e virginica se sobrepõem. Com K = 4 e 5 o SSE continua caindo, mas só porque grupos menores sempre ficam mais compactos. As espécies são divididas em pedaços e o erro sobe.

Na **Diabetes** o agrupamento não recupera as classes. Com K = 2 os grupos têm 515 e 253 pacientes, e o grupo associado aos diabéticos tem 133 diabéticos e 120 saudáveis, quase meio a meio. O erro de 33% é praticamente o que se teria colocando todos na classe “negativo” (35%). Os centroides mostram o que o K-Means separou: o grupo menor tem pacientes mais velhas (46 anos em média, contra 27) e com mais gestações (7,4 contra 2,1). A idade pesa no diagnóstico, mas não o define, e por isso os grupos misturam as duas classes. Com mais grupos o SSE cai, mas os grupos se misturam ainda mais em relação às classes. Um agrupamento bom não precisa coincidir com a classe que interessa.

Water Treatment Plant. A base tem 527 dias de medições diárias de uma estação de tratamento de esgoto, com 38 atributos: vazão, pH, DBO, DQO, sólidos e condutividade na entrada, em cada etapa (decantador primário e secundário) e na saída, além do rendimento de cada etapa. Não há classe. Rodamos o K-Means de K = 2 a K = 10 e olhamos o SSE.

Tabela 2. SSE e tamanho dos grupos na Water Treatment.

K	SSE	Tamanho dos grupos
2	252,2	234, 293
3	232,9	248, 82, 197
4	219,8	242, 79, 199, 7
5	210,2	109, 13, 181, 7, 217
6	198,4	86, 14, 111, 7, 167, 142
8	180,9	49, 10, 85, 8, 127, 115, 63, 70
10	172,2	48, 5, 77, 8, 107, 60, 45, 42, 48, 87

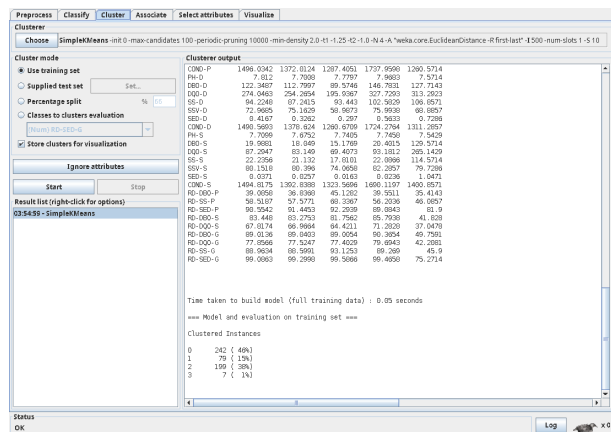
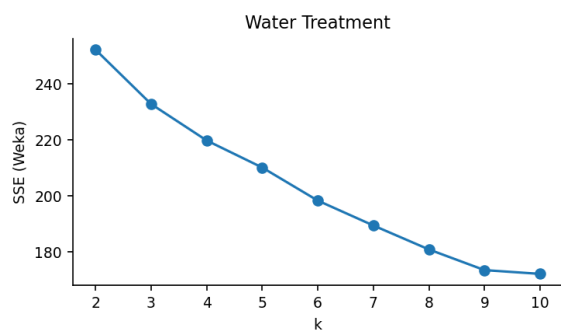


Figura 2. Curva do SSE (esquerda) e o resultado com K = 4 no Weka (direita).

O SSE cai sem um “cotovelo” bem marcado. As maiores quedas são de 2 para 3 (19 pontos) e de 3 para 4 (13 pontos), e a partir daí ficam parecidas. Também calculamos a silhueta no script, com o scikit-learn, e ela ficou baixa para todos os K: 0,148 com K = 2, 0,108 com K = 4 e cerca de 0,10 daí em diante. Isso indica que os dias não formam grupos bem separados: a operação da estação varia de forma contínua. Então escolhemos K olhando também se os grupos fazem sentido. Com **K = 4**, os centroides mostram quatro situações diferentes:

- **242 dias de operação normal**, com valores perto da média em tudo;
- **79 dias com vazão alta** (42 mil contra 37 mil) e muitos sólidos na entrada (351 contra 227), mas com DBO, DQO e condutividade baixas. É esgoto diluído, o que sugere dias de chuva;
- **199 dias com esgoto mais concentrado**: DBO, DQO e condutividade altas na entrada, provavelmente dias secos;
- **7 dias com falha no tratamento**: a DBO na saída é de 130 contra 20 nos outros grupos, os sólidos na saída vão de 22 para 115 e o rendimento global cai de 89% para 50%. O problema está no decantador secundário, cujo rendimento de DBO cai de 83% para 42%.

O melhor valor que encontramos foi **K = 4**. Com K = 2 ou 3 os dias de falha ficam escondidos dentro dos outros grupos, e com K maior que 4 os grupos de operação normal passam a ser divididos sem ganhar interpretação nova. Numa base não supervisionada como essa, o K-Means serve para descrever os modos de operação da estação e, principalmente, para achar os dias anormais. O grupo de 7 dias é o tipo de resultado que interessa ao operador, porque mostra quando e em que etapa a estação falhou.

Questão 2

Considere o seguinte conjunto de exemplos não rotulados. Utilize o algoritmo K-Means para encontrar as 3 classes existentes. Calcule e mostre a posição dos centroides durante todas as iterações do algoritmo.

Resposta. Pela figura da lista os 17 pontos formam três regiões: em cima à esquerda (pontos 1 a 7), no meio à direita (8 a 11) e embaixo à esquerda (12 a 17). Escolhemos como centroides iniciais um ponto de cada região: o ponto 1 (1,9; 7,3), o ponto 9 (5; 3,2) e o ponto 15 (0,8; 2). Usamos a distância euclidiana. Em cada iteração, cada ponto vai para o centroide mais próximo, e depois cada centroide vai para a média dos seus pontos:

```
novos = [argmin(dist(x, c) for c in centros) for x in pontos]
centros[j] = media(pontos do grupo j)
```

Tabela 3. Centroides e grupos em cada iteração.

Iteração	C1	C2	C3
início	(1,900; 7,300)	(5,000; 3,200)	(0,800; 2,000)
1	(2,500; 6,717) {1, 2, 3, 4, 5, 6}	(4,500; 3,480) {7, 8, 9, 10, 11}	(1,367; 2,150) {12, 13, 14, 15, 16, 17}
2	(2,629; 6,500) {1, 2, 3, 4, 5, 6, 7}	(4,775; 3,050) {8, 9, 10, 11}	(1,367; 2,150) {12, 13, 14, 15, 16, 17}
3	nenhum ponto muda de grupo: convergiu		

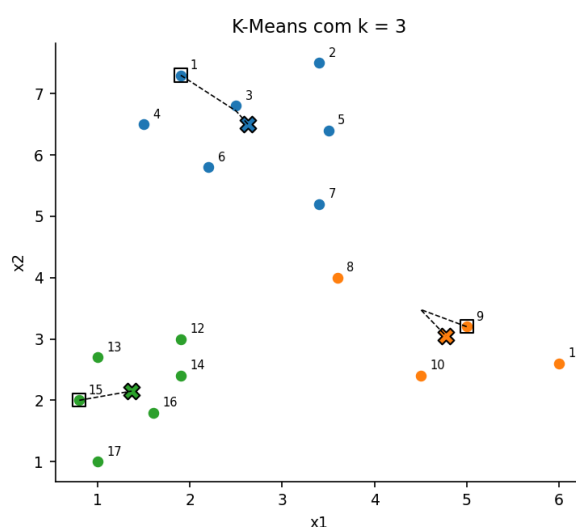


Figura 3. Grupos finais. Os quadrados são os centroides iniciais, os X os finais e a linha tracejada o caminho de cada centroide.

Na primeira iteração o ponto 7 (3,4; 5,2) ficou com o grupo do meio, porque estava mais perto do ponto 9 que do ponto 1, por muito pouco (2,56 contra 2,58). Quando C1 desceu para a média do grupo de cima, o ponto 7 passou a ficar mais perto dele e mudou de grupo na segunda iteração. Na terceira nada mudou e o algoritmo parou. Os grupos finais são exatamente as três regiões da figura, com SSE de 16,2.

Questão 3

Considerando a distribuição de pontos abaixo, faça: a. Aplique o K-Means utilizando as seguintes centroides iniciais $K1 = P2$ e $K2 = P8$ e a distância de Minkowski. b. Selecione um novo par de centroides iniciais de modo que o resultado apresente dois clusters diferentes do exercício anterior, utilize também a distância de Manhattan.

Resposta. A distância de Minkowski é $d = (\sum |a_i - b_i|^p)^{1/p}$. Com $p = 1$ ela é a Manhattan e com $p = 2$ a euclidiana, então no item a usamos $p = 3$, para ser diferente da Manhattan do item b.

a) Com $K1 = P2$ (3,1) e $K2 = P8$ (8,4):

- iteração 1: P1, P2 e P3 ficam com K1 e os outros seis com K2. Os centroides vão para $K1 = (3,000; 2,000)$ e $K2 = (9,667; 5,000)$;
- iteração 2: nenhum ponto muda de grupo, e o algoritmo para.

Resultado: {P1, P2, P3} e {P4, P5, P6, P7, P8, P9}.

b) Antes de escolher, testamos no script quais pares de centroides iniciais dariam grupos finais diferentes. Testamos os 36 pares possíveis de pontos da base e depois uma grade de posições quaisquer (x de 0 a 14 e y de 0 a 8), sempre com a distância de Manhattan. **Nenhum par terminou numa partição diferente:** todos convergiram para {P1, P2, P3} e {P4, ..., P9}.

Escolhemos então $K1 = (5, 4)$ e $K2 = (12, 5)$, um centroide mais à esquerda do meio e outro em cima dos pontos da direita, para mostrar o que acontece:

Tabela 4. Iterações com $K1 = (5, 4)$, $K2 = (12, 5)$ e distância de Manhattan.

It.	Grupo 1	Grupo 2	Novos centroides
1	{P1, P2, P3, P7, P8, P9}	{P4, P5, P6}	(5,333; 3,500) e (11,667; 5,000)
2	{P1, P2, P3, P7, P8}	{P4, P5, P6, P9}	(4,800; 3,000) e (10,750; 5,250)
3	{P1, P2, P3}	{P4, ..., P9}	(3,000; 2,000) e (9,667; 5,000)
4	nenhum ponto muda de grupo		

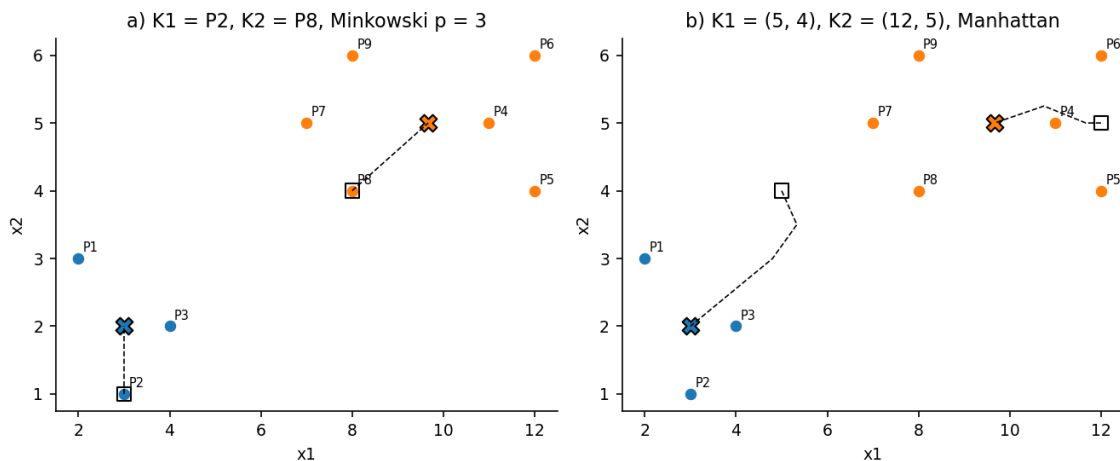


Figura 4. Grupos finais dos itens a e b e o caminho dos centroides.

Nas duas primeiras iterações os grupos são diferentes dos do item a: primeiro {P1, P2, P3, P7, P8, P9} contra {P4, P5, P6}, depois o P9 passa para a direita. Mas deixando o algoritmo chegar ao fim, ele volta à mesma partição.

A explicação está na forma dos dados. P1, P2 e P3 ficam em volta de (3, 2) e os outros seis em volta de (9,7; 5), e a distância entre os dois grupos é muito maior que a distância dentro de cada um. O K-Means só para quando todo ponto está mais perto do próprio centroide, e para dois grupos essa condição só é satisfeita por essa partição. Qualquer outra divisão tem algum ponto mais perto do centroide do outro grupo, que então muda de lado, como aconteceu com P9 e depois com P7 e P8. Com esses dados e $K = 2$, dois clusters finais diferentes só apareceriam parando o algoritmo antes de convergir (limitando o número de iterações a 1 ou 2, por exemplo) ou mudando o K. Com $K = 3$, os pontos da direita se dividem em {P7, P8, P9} e {P4, P5, P6}.

Apêndice A. Código

```
pip install numpy matplotlib scikit-learn
python converter_water.py water-treatment.data
python solucao.py
```

converter_water.py

```
# Converte a base Water Treatment Plant (UCI) para .arff
# Uso: python converter_water.py water-treatment.data
# A primeira coluna é a data do registro e fica de fora.

import sys

NOMES = ["Q-E", "ZN-E", "PH-E", "DBO-E", "DQO-E", "SS-E", "SSV-E", "SED-E",
         "COND-E", "PH-P", "DBO-P", "SS-P", "SSV-P", "SED-P", "COND-P",
         "PH-D", "DBO-D", "DQO-D", "SS-D", "SSV-D", "SED-D", "COND-D",
         "PH-S", "DBO-S", "DQO-S", "SS-S", "SSV-S", "SED-S", "COND-S",
         "RD-DBO-P", "RD-SS-P", "RD-SED-P", "RD-DBO-S", "RD-DQO-S",
         "RD-DBO-G", "RD-DQO-G", "RD-SS-G", "RD-SED-G"]

texto = open(sys.argv[1]).read()
# alguns registros estão quebrados em duas linhas; junta tudo e separa
# pelo começo de cada data, que sempre é "D-"
campos = texto.replace("\n", "").split("D-")[1:]
linhas = []
for c in campos:
    valores = c.rstrip(",").split(",")[1:]
    if len(valores) != len(NOMES):
        sys.exit(f"registro com {len(valores)} valores: {c[:40]}")
    linhas.append(",".join(valores))

with open("water-treatment.arff", "w") as f:
    f.write("@relation water-treatment\n\n")
    for n in NOMES:
        f.write(f"@attribute {n} numeric\n")
    f.write("\n@data\n")
    f.write("\n".join(linhas) + "\n")
print(f"{len(linhas)} registros, {len(NOMES)} atributos")
```

solucao.py

```
# Lista 10 - K-Means
# Sistemas Inteligentes Aplicados - UTFPR
# Uso: python solucao.py
# (a questão 1 foi feita no Weka; ver saidas/weka_kmeans_tabela.txt)

import os
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

PASTA = os.path.dirname(os.path.abspath(__file__))
SAIDA = os.path.join(PASTA, "saidas")
os.makedirs(SAIDA, exist_ok=True)
plt.rcParams.update({"font.size": 9, "axes.spines.top": False,
                    "axes.spines.right": False})

def minkowski(a, b, p):
    return np.sum(np.abs(a - b) ** p) ** (1 / p)

def kmeans(pontos, centros, p, mostrar=True, max_iter=50):
    centros = np.array(centros, dtype=float)
    historico = [centros.copy()]
    grupos = None
    for it in range(1, max_iter + 1):
        # cada ponto vai para o centroide mais próximo
```

```

novos = np.array([np.argmin([minkowski(x, c, p) for c in centros])
                  for x in pontos])
if grupos is not None and np.array_equal(novos, grupos):
    if mostrar:
        print(f"  iteração {it}: nenhum ponto mudou de grupo, parou")
    break
grupos = novos
# o centróide vai para a média dos seus pontos
for j in range(len(centros)):
    if np.any(grupos == j):
        centros[j] = pontos[grupos == j].mean(axis=0)
historico.append(centros.copy())
if mostrar:
    txt = []
    for j in range(len(centros)):
        membros = [str(i + 1) for i in np.where(grupos == j)[0]]
        txt.append(f"C{j + 1}={centros[j][0]:.3f}, {centros[j][1]:.3f}")
        f" {{{', '.join(membros)}}}"
    print(f"  iteração {it}: " + " ".join(txt))
return grupos, historico

def desenhar(ax, pontos, grupos, historico, rotulos, titulo):
    cores = ["tab:blue", "tab:orange", "tab:green"]
    for j in range(len(historico[0])):
        ax.scatter(*pontos[grupos == j].T, color=cores[j], s=30)
        trilha = np.array([h[j] for h in historico])
        ax.plot(*trilha.T, "k--", lw=0.8)
        ax.scatter(*trilha[0], marker="s", facecolor="none", edgecolor="k", s=60)
        ax.scatter(*trilha[-1], marker="X", color=cores[j], edgecolor="k", s=90)
    for i, (x, y) in enumerate(pontos):
        ax.annotate(rotulos[i], (x, y), (x + 0.1, y + 0.1), fontsize=7)
    ax.set_title(titulo)
    ax.set_xlabel("x1")
    ax.set_ylabel("x2")

# questao 2
print("[2] 17 pontos, 3 grupos, distância euclidiana")
q2 = np.array([[1.9, 7.3], [3.4, 7.5], [2.5, 6.8], [1.5, 6.5], [3.5, 6.4],
               [2.2, 5.8], [3.4, 5.2], [3.6, 4], [5, 3.2], [4.5, 2.4],
               [6, 2.6], [1.9, 3], [1, 2.7], [1.9, 2.4], [0.8, 2], [1.6, 1.8],
               [1, 1]])
# centroides iniciais: um ponto de cada região da figura (1, 9 e 15)
inicio = q2[[0, 8, 14]]
print("  iniciais: pontos 1, 9 e 15")
g2, h2 = kmeans(q2, inicio, 2)
sse = sum(np.sum((q2[g2 == j] - h2[-1][j]) ** 2) for j in range(3))
print(f"  SSE final = {sse:.3f}")

fig, ax = plt.subplots(figsize=(5, 4.5))
desenhar(ax, q2, g2, h2, [str(i + 1) for i in range(17)],
         "K-Means com k = 3")
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q2_kmeans.png"), dpi=150)
plt.close(fig)

# questao 3
print("\n[3a] P1 a P9, K1 = P2, K2 = P8, Minkowski com p = 3")
q3 = np.array([[2, 3], [3, 1], [4, 2], [11, 5], [12, 4], [12, 6], [7, 5],
               [8, 4], [8, 6]], dtype=float)
nomes = [f"P{i + 1}" for i in range(9)]
g3a, h3a = kmeans(q3, q3[[1, 7]], 3)

print("\n[3b] procurando centroides iniciais que dêem outra partição")
# primeiro testa todos os pares de pontos da base como centroides
final_a = {frozenset(np.where(g3a == j)[0]) for j in range(2)}
iguais = 0
for i in range(9):
    for j in range(i + 1, 9):
        g, _ = kmeans(q3, q3[[i, j]], 1, mostrar=False)
        final = {frozenset(np.where(g == k)[0]) for k in range(2)}
        iguais += final == final_a
print(f"  com pares de pontos da base: {iguais} de 36 terminam igual ao item a")

```

```

# depois testa centroides em posições quaisquer de uma grade
achou = None
for x1 in np.arange(0, 15, 1.0):
    for y1 in np.arange(0, 9, 1.0):
        for x2 in np.arange(0, 15, 1.0):
            for y2 in np.arange(0, 9, 1.0):
                g, _ = kmeans(q3, [[x1, y1], [x2, y2]], 1, mostrar=False)
                if len(set(g)) < 2:
                    continue
                final = {frozenset(np.where(g == k)[0]) for k in range(2)}
                if final != final_a:
                    achou = ([x1, y1], [x2, y2])
                    break
            if achou:
                break
        if achou:
            break
    if achou:
        break
if achou:
    print(f" par da grade com outra partição: {achou}")
else:
    print(" nenhum par da grade (x de 0 a 14, y de 0 a 8) dá outra partição")

# par escolhido para o relatório: um centroide de cada lado da base
print(" K1 = (5, 4) e K2 = (12, 5), Manhattan")
g3b, h3b = kmeans(q3, [[5, 4], [12, 5]], 1)

fig, ax = plt.subplots(1, 2, figsize=(9, 3.8))
desenhar(ax[0], q3, g3a, h3a, nomes, "a) K1 = P2, K2 = P8, Minkowski p = 3")
desenhar(ax[1], q3, g3b, h3b, nomes, "b) K1 = (5, 4), K2 = (12, 5), Manhattan")
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q3_kmeans.png"), dpi=150)
plt.close(fig)

# gráfico do cotovelo da base Water Treatment, com os números do Weka
tabela = os.path.join(SAIDA, "weka_kmeans_tabela.txt")
if os.path.exists(tabela):
    ks = list(range(2, 11))
    sse = []
    for linha in open(tabela):
        if linha.startswith("water"):
            sse.append(float(linha.split("sse=")[1].split()[0]))
    fig, ax = plt.subplots(figsize=(5, 3))
    ax.plot(ks, sse, "o-")
    ax.set_xlabel("k")
    ax.set_ylabel("SSE (Weka)")
    ax.set_title("Water Treatment")
    fig.tight_layout()
    fig.savefig(os.path.join(SAIDA, "q1_cotovelo_water.png"), dpi=150)
    plt.close(fig)

# silhueta da Water Treatment para cada k, com os mesmos cuidados do Weka:
# faltantes trocados pela média e atributos normalizados de 0 a 1
arff = os.path.join(PASTA, "water-treatment.arff")
if os.path.exists(arff):
    from sklearn.cluster import KMeans
    from sklearn.metrics import silhouette_score
    print("\n[1] silhueta da Water Treatment")
    dados = []
    for linha in open(arff):
        if linha[0].isdigit():
            dados.append([float(v) if v != "?" else np.nan
                           for v in linha.strip().split(",")])
    X = np.array(dados)
    medias = np.nanmean(X, axis=0)
    faltando = np.where(np.isnan(X))
    X[faltando] = medias[faltando[1]]
    X = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
    for k in range(2, 11):
        notas = [silhouette_score(X, KMeans(k, n_init=10, random_state=r)
                                   .fit_predict(X)) for r in range(3)]

```



```
print(f" k={k}: silhueta média {np.mean(notas):.3f}")
```